

Structured Computer Architecture
Chapter 2.8: Combinational Circuits
Zero-Order Systems (0-OS)
An Order-Based Approach

John Glossner and Gheorghe M. Ștefan

Based on the textbook

September 14, 2025

Outline

Introduction to Zero-Order Systems

Boolean Algebra

Truth Tables

Logic Gates and Digital Circuits

Combinational Building Blocks

Circuit Optimization and Minimization

Advanced Combinational Concepts

Chapter Summary

Learning Objectives

By the end of this chapter, you will be able to:

- ▶ Understand the fundamentals of Boolean algebra and its applications
- ▶ Construct and analyze truth tables for logical operations
- ▶ Design and optimize combinational logic circuits
- ▶ Apply Boolean laws and identities for circuit simplification
- ▶ Understand the relationship between mathematical logic and digital circuits
- ▶ Recognize the historical development of Boolean algebra in computing

What are Zero-Order Systems (0-OS)?

- ▶ **Definition:** Digital systems with **no feedback loops**
 - ▶ Output depends *only* on current input values
 - ▶ No memory or state retention
 - ▶ Purely combinational behavior
- ▶ **Characteristics:**
 - ▶ Immediate response to input changes
 - ▶ No temporal dependencies
 - ▶ Foundation for all digital logic
 - ▶ Building blocks for higher-order systems
- ▶ **Examples:** Logic gates, decoders, multiplexers, adders

Order-Based System Hierarchy

- ▶ **0-OS (Zero-Order):** Combinational circuits
 - ▶ No loops, no memory
 - ▶ Output = $f(\text{current inputs})$
- ▶ **1-OS (First-Order):** Memory circuits
 - ▶ One feedback loop
 - ▶ Can store state
- ▶ **2-OS (Second-Order):** Automata
 - ▶ Two feedback loops
 - ▶ Autonomous behavior
- ▶ **3-OS (Third-Order):** Processors
 - ▶ Three feedback loops
 - ▶ Complex computational behavior

Historical Background

- ▶ **George Boole (1815-1864):**

- ▶ Published "An Investigation of the Laws of Thought" (1854)
- ▶ Formalized logical reasoning using algebraic structures
- ▶ Created symbolic language for logic

- ▶ **Claude Shannon (1916-2001):**

- ▶ Master's thesis: "A Symbolic Analysis of Relay and Switching Circuits" (1938)
- ▶ Connected Boolean algebra to electrical circuits
- ▶ Foundation of digital circuit design
- ▶ Enabled modern computer development

- ▶ **Impact:** Transformed abstract mathematics into practical engineering tool

Boolean Algebra Fundamentals

- ▶ **Binary Set:** $B = \{0, 1\}$

- ▶ 0 represents **false**
- ▶ 1 represents **true**

- ▶ **Basic Operations:**

- ▶ **AND** ($\wedge$): $A \wedge B = 1$ iff both $A = 1$ and $B = 1$
- ▶ **OR** ($\vee$): $A \vee B = 1$ if at least one of A or B equals 1
- ▶ **NOT** ($\neg$): $\neg A = 1$ if $A = 0$, and $\neg A = 0$ if $A = 1$

- ▶ **Physical Interpretation:**

- ▶ 0 = Low voltage, switch open, false
- ▶ 1 = High voltage, switch closed, true

Boolean Laws and Identities

Commutative Laws:

$$A \vee B = B \vee A \quad (1)$$

$$A \wedge B = B \wedge A \quad (2)$$

Associative Laws:

$$A \vee (B \vee C) = (A \vee B) \vee C \quad (3)$$

$$A \wedge (B \wedge C) = (A \wedge B) \wedge C \quad (4)$$

Distributive Laws:

$$A \wedge (B \vee C) = (A \wedge B) \vee (A \wedge C) \quad (5)$$

$$A \vee (B \wedge C) = (A \vee B) \wedge (A \vee C) \quad (6)$$

Identity Laws:

$$A \vee 0 = A \quad (7)$$

$$A \wedge 1 = A \quad (8)$$

Complement Laws:

$$A \vee \neg A = 1 \quad (9)$$

$$A \wedge \neg A = 0 \quad (10)$$

De Morgan's Laws:

$$\neg(A \vee B) = \neg A \wedge \neg B \quad (11)$$

$$\neg(A \wedge B) = \neg A \vee \neg B \quad (12)$$

Boolean Algebra Applications

▶ **Circuit Simplification:**

- ▶ Reduce number of gates required
- ▶ Minimize cost and complexity
- ▶ Improve performance and reliability

▶ **Simplification Methods:**

- ▶ **Boolean identities:** Algebraic manipulation
- ▶ **Karnaugh maps:** Graphical simplification method
- ▶ **Quine-McCluskey algorithm:** Tabular minimization approach

▶ **Logic Conversion:**

- ▶ Convert between AND-OR and NAND-NOR logic
- ▶ Transform to preferred gate types
- ▶ Optimize for specific technologies

Truth Table Fundamentals

- ▶ **Definition:** Systematic representation of logical expressions
 - ▶ Lists all possible input combinations
 - ▶ Shows corresponding output values
 - ▶ Complete behavioral specification
- ▶ **Structure Components:**
 - ▶ **Input Variables:** Columns for logical variables ($A, B, \dots, N$)
 - ▶ **Output Values:** Results of logical operations
 - ▶ **Rows:** 2^n combinations for n variables
- ▶ **Applications:**
 - ▶ Circuit analysis and design
 - ▶ Logic verification
 - ▶ Boolean function specification

Basic Logic Operations - Truth Tables

AND Operation:

A	B	$A \wedge B$
0	0	0
0	1	0
1	0	0
1	1	1

OR Operation:

A	B	$A \vee B$
0	0	0
0	1	1
1	0	1
1	1	1

NOT Operation:

A	$\neg A$
0	1
1	0

XOR (Exclusive OR):

A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

NAND (NOT AND):

A	B	$\neg(A \wedge B)$
0	0	1
0	1	1
1	0	1
1	1	0

Advanced Logic Operations

NOR (NOT OR):

A	B	$\neg(A \vee B)$
0	0	1
0	1	0
1	0	0
1	1	0

XNOR (Equivalence):

A	B	$A \odot B$
0	0	1
0	1	0
1	0	0
1	1	1

Key Properties:

- ▶ **NAND**: Universal gate (can implement any Boolean function)
- ▶ **NOR**: Also universal gate
- ▶ **XOR**: Useful for parity checking and arithmetic
- ▶ **XNOR**: Equivalence testing and error detection

Truth Table Applications

▶ **Circuit Analysis:**

- ▶ Verify circuit behavior
- ▶ Compare expected vs. actual outputs
- ▶ Debug logic errors

▶ **Function Specification:**

- ▶ Define required circuit behavior
- ▶ Specify input-output relationships
- ▶ Document design requirements

▶ **Logic Synthesis:**

- ▶ Convert truth tables to Boolean expressions
- ▶ Generate minimal circuit implementations
- ▶ Optimize for area, speed, or power

Physical Implementation of Logic Gates

▶ **Transistor-Level Implementation:**

- ▶ CMOS technology (Complementary Metal-Oxide-Semiconductor)
- ▶ NMOS and PMOS transistors
- ▶ Low power consumption and high noise immunity

▶ **Gate Characteristics:**

- ▶ **Propagation delay:** Time for output to respond to input change
- ▶ **Fan-in:** Number of inputs a gate can accept
- ▶ **Fan-out:** Number of gates an output can drive
- ▶ **Noise margin:** Tolerance to signal variations

▶ **Performance Considerations:**

- ▶ Speed vs. power trade-offs
- ▶ Area optimization
- ▶ Signal integrity

Combinational Circuit Design Process

1. Problem Specification:

- ▶ Define input and output requirements
- ▶ Specify functional behavior
- ▶ Identify constraints (timing, power, area)

2. Truth Table Construction:

- ▶ List all input combinations
- ▶ Determine corresponding outputs
- ▶ Verify completeness and correctness

3. Boolean Expression Derivation:

- ▶ Sum-of-products (SOP) form
- ▶ Product-of-sums (POS) form
- ▶ Choose optimal representation

4. Circuit Implementation:

- ▶ Select appropriate gates
- ▶ Optimize for design goals
- ▶ Verify functionality

Half Adder Circuit

Function: Adds two single bits

- ▶ Inputs: A , B
- ▶ Outputs: Sum (S), Carry (C)

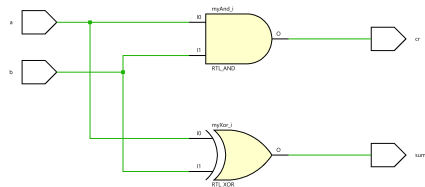
Truth Table:

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Boolean Expressions:

$$S = A \oplus B \quad (13)$$

$$C = A \wedge B \quad (14)$$



Implementation:

- ▶ XOR gate for sum
- ▶ AND gate for carry
- ▶ Fundamental building block

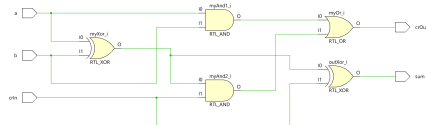
Full Adder Circuit

Function: Adds three bits (including carry-in)

- Inputs: A , B , C_{in}
- Outputs: Sum (S), Carry-out (C_{out})

Truth Table:

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



Boolean Expressions:

$$S = A \oplus B \oplus C_{in} \quad (15)$$

$$C_{out} = AB + C_{in}(A \oplus B) \quad (16)$$

Decoders

- ▶ **Function:** Convert binary code to one-hot output
 - ▶ n inputs produce 2^n outputs
 - ▶ Only one output is active (high) at a time
 - ▶ Used for address decoding and selection
- ▶ **2-to-4 Decoder Example:**
 - ▶ Inputs: A_1, A_0
 - ▶ Outputs: Y_0, Y_1, Y_2, Y_3
 - ▶ $Y_i = 1$ when input equals i in binary
- ▶ **Applications:**
 - ▶ Memory address decoding
 - ▶ Instruction decoding in processors
 - ▶ Device selection in systems
 - ▶ Seven-segment display drivers

Multiplexers (Data Selectors)

- ▶ **Function:** Select one of multiple inputs based on control signals
 - ▶ 2^n data inputs, n select inputs
 - ▶ One output that equals selected input
 - ▶ Digital equivalent of rotary switch
- ▶ **4-to-1 Multiplexer:**
 - ▶ Data inputs: D_0, D_1, D_2, D_3
 - ▶ Select inputs: S_1, S_0
 - ▶ Output: $Y = D_i$ where $i = S_1S_0$
- ▶ **Applications:**
 - ▶ Data routing and switching
 - ▶ Function generation
 - ▶ Parallel-to-serial conversion
 - ▶ Processor data paths

Demultiplexers (Data Distributors)

- ▶ **Function:** Route single input to one of multiple outputs
 - ▶ One data input, n select inputs
 - ▶ 2^n outputs, only one receives data
 - ▶ Inverse operation of multiplexer
- ▶ **1-to-4 Demultiplexer:**
 - ▶ Data input: D
 - ▶ Select inputs: S_1, S_0
 - ▶ Outputs: Y_0, Y_1, Y_2, Y_3
 - ▶ $Y_i = D$ when $i = S_1 S_0$, others = 0
- ▶ **Applications:**
 - ▶ Data distribution
 - ▶ Serial-to-parallel conversion
 - ▶ Address decoding with enable
 - ▶ Communication systems

Encoders

- ▶ **Function:** Convert one-hot input to binary code
 - ▶ 2^n inputs produce n outputs
 - ▶ Inverse operation of decoder
 - ▶ Assumes only one input is active
- ▶ **Priority Encoder:**
 - ▶ Handles multiple active inputs
 - ▶ Encodes highest priority input
 - ▶ Includes valid output signal
 - ▶ More practical than simple encoder
- ▶ **Applications:**
 - ▶ Keyboard encoding
 - ▶ Interrupt priority encoding
 - ▶ Data compression
 - ▶ Position encoding

Karnaugh Maps (K-Maps)

- ▶ **Purpose:** Graphical method for Boolean function minimization

- ▶ Visual representation of truth table
- ▶ Adjacent cells differ by one variable
- ▶ Groups of 1s indicate simplified terms

- ▶ **Procedure:**

1. Plot function on K-map grid
2. Identify groups of adjacent 1s
3. Groups must be powers of 2 (1, 2, 4, 8, ...)
4. Write simplified expression from groups

- ▶ **Advantages:**

- ▶ Intuitive visual approach
- ▶ Guaranteed minimal result for small functions
- ▶ Easy to identify don't-care conditions

Quine-McCluskey Algorithm

- ▶ **Purpose:** Systematic tabular minimization method
 - ▶ Algorithmic approach to Boolean minimization
 - ▶ Handles any number of variables
 - ▶ Computer-implementable procedure

- ▶ **Steps:**
 1. List all minterms in binary
 2. Find prime implicants by combining terms
 3. Construct prime implicant table
 4. Select minimal set of prime implicants

- ▶ **Advantages:**
 - ▶ Systematic and complete
 - ▶ Handles large numbers of variables
 - ▶ Suitable for computer automation
 - ▶ Finds all minimal solutions

Design Optimization Goals

- ▶ **Area Minimization:**
 - ▶ Reduce number of gates
 - ▶ Minimize chip area and cost
 - ▶ Important for high-volume production
- ▶ **Speed Optimization:**
 - ▶ Minimize propagation delay
 - ▶ Reduce critical path length
 - ▶ May require more gates for parallelism
- ▶ **Power Minimization:**
 - ▶ Reduce switching activity
 - ▶ Use low-power gate designs
 - ▶ Critical for mobile and embedded systems
- ▶ **Trade-offs:**
 - ▶ Speed vs. area vs. power
 - ▶ Design constraints determine priorities
 - ▶ Technology-dependent optimizations

Hazards and Glitches

▶ **Static Hazards:**

- ▶ Output momentarily changes when it should remain constant
- ▶ Caused by unequal propagation delays
- ▶ Can be eliminated by adding redundant terms

▶ **Dynamic Hazards:**

- ▶ Output changes multiple times during transition
- ▶ More complex to analyze and eliminate
- ▶ Usually avoided through careful design

▶ **Prevention Techniques:**

- ▶ Add consensus terms to cover hazards
- ▶ Use synchronous design methodology
- ▶ Careful timing analysis and verification
- ▶ Buffer insertion for delay matching

Technology Mapping

▶ **Gate Library Constraints:**

- ▶ Available gates in target technology
- ▶ Drive strength and loading considerations
- ▶ Area and delay characteristics

▶ **NAND/NOR Implementation:**

- ▶ Universal gates can implement any function
- ▶ Often preferred in CMOS technology
- ▶ Requires function transformation

▶ **Optimization Considerations:**

- ▶ Technology-specific optimizations
- ▶ Wire delay vs. gate delay
- ▶ Power and thermal constraints
- ▶ Manufacturing yield considerations

Arithmetic Circuits

▶ **Multi-bit Adders:**

- ▶ Ripple-carry adder: Simple but slow
- ▶ Carry-lookahead adder: Faster but more complex
- ▶ Carry-select adder: Speed-area trade-off

▶ **Subtraction Circuits:**

- ▶ Two's complement representation
- ▶ Subtraction as addition of complement
- ▶ Overflow detection logic

▶ **Comparison Circuits:**

- ▶ Magnitude comparators
- ▶ Equality detection
- ▶ Greater-than/less-than logic

Key Concepts Review

▶ **Boolean Algebra Foundations:**

- ▶ Mathematical basis for digital logic
- ▶ Laws and identities for simplification
- ▶ Historical development from Boole to Shannon

▶ **Truth Tables and Logic Operations:**

- ▶ Systematic representation of logical functions
- ▶ Basic and advanced logic operations
- ▶ Circuit analysis and verification tool

▶ **Combinational Circuit Design:**

- ▶ Systematic design methodology
- ▶ Building blocks: decoders, multiplexers, adders
- ▶ Optimization techniques and trade-offs

Practical Applications

▶ **Digital System Building Blocks:**

- ▶ Foundation for all digital circuits
- ▶ Essential components in processors
- ▶ Building blocks for memory systems

▶ **Design Methodology:**

- ▶ Specification to implementation flow
- ▶ Optimization for different goals
- ▶ Verification and testing approaches

▶ **Technology Considerations:**

- ▶ Physical implementation constraints
- ▶ Performance optimization techniques
- ▶ Power and area trade-offs

Connection to Higher-Order Systems

- ▶ **Foundation for 1-OS (Memory Circuits):**

- ▶ Combinational logic + feedback = memory
- ▶ State storage and retention
- ▶ Clocking and synchronization

- ▶ **Building Blocks for 2-OS (Automata):**

- ▶ Control logic implementation
- ▶ State transition logic
- ▶ Output generation circuits

- ▶ **Components of 3-OS (Processors):**

- ▶ Arithmetic and logic units
- ▶ Instruction decoding logic
- ▶ Data path components

Questions and Discussion

Questions?

Discussion Topics:

- ▶ How do Boolean laws enable circuit optimization?
- ▶ What are the trade-offs between different adder architectures?
- ▶ Why are NAND and NOR gates considered universal?
- ▶ How do combinational circuits form the foundation for sequential systems?

Next: Chapter 2.9 - Memory Circuits: First-Order Systems (1-OS)